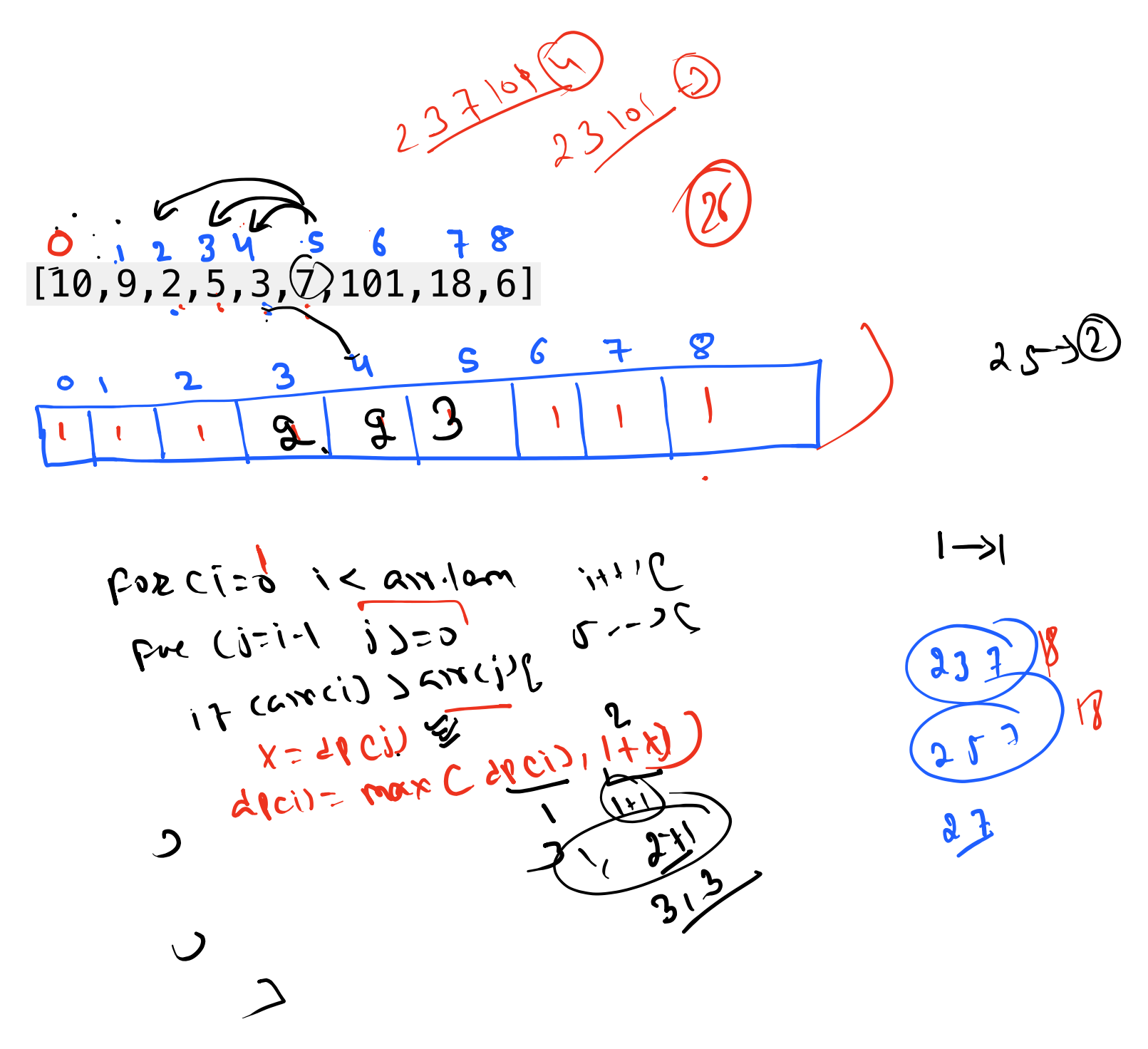
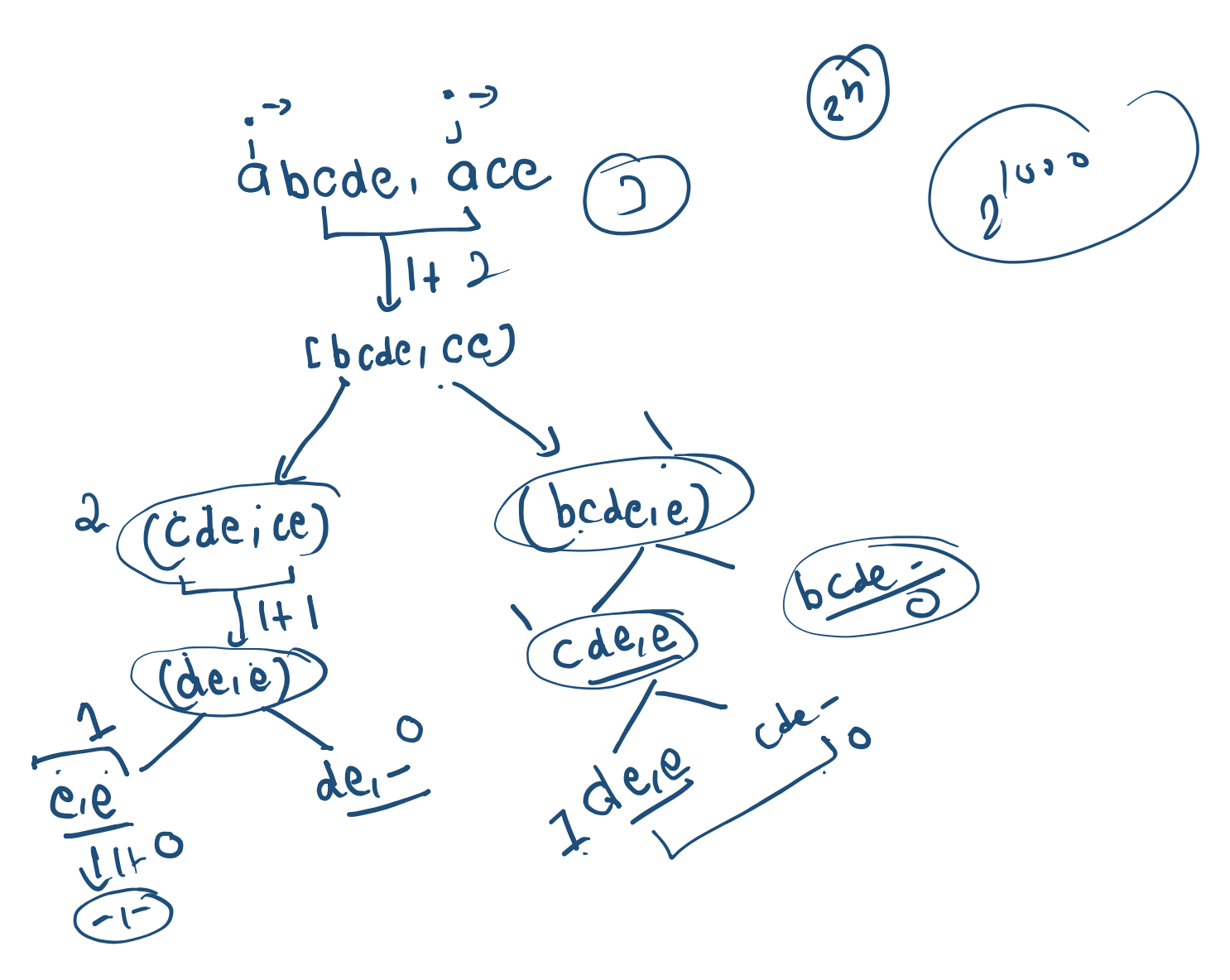
# Lec-41

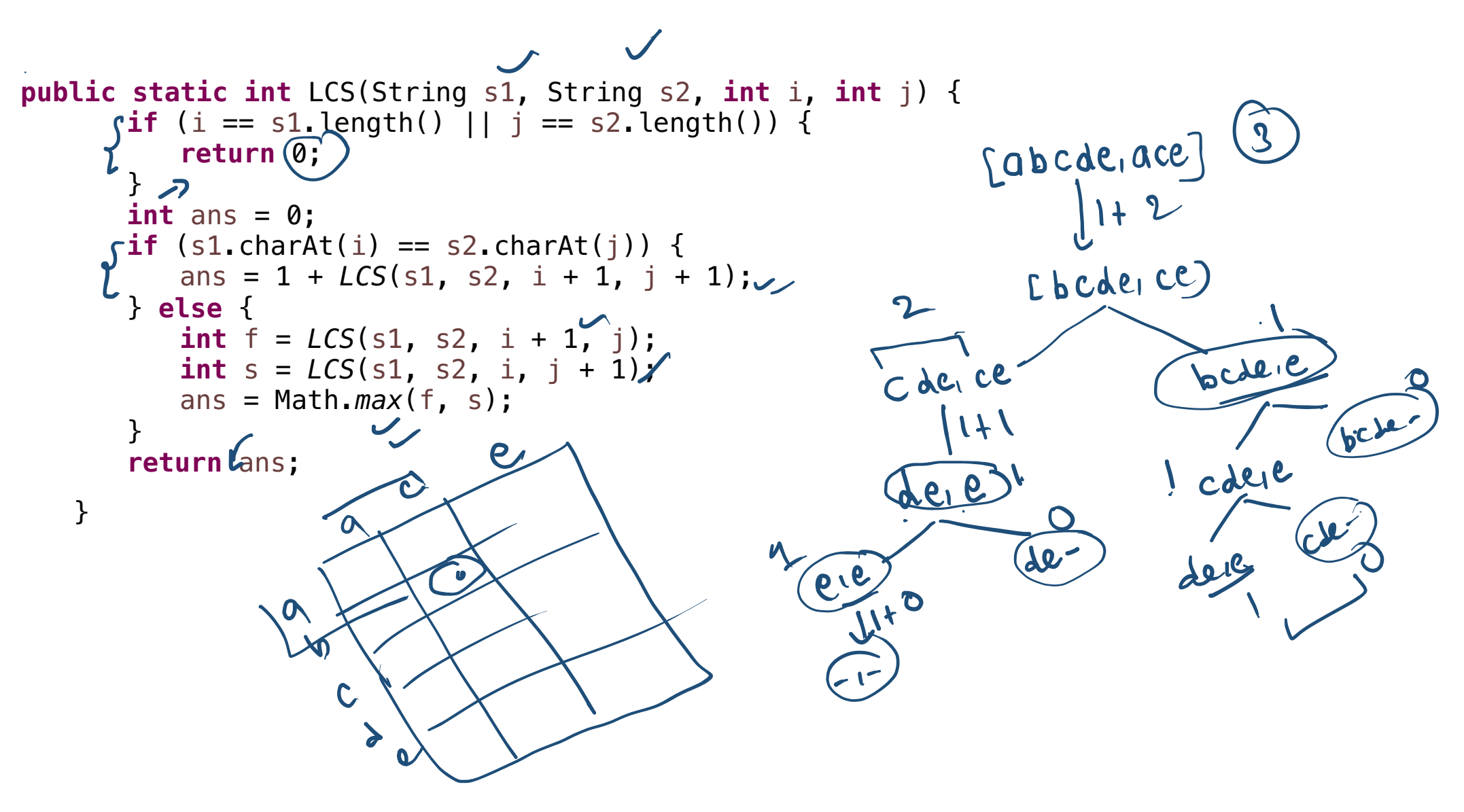
12/04/26 12:42 PM



[10,9,2,5,3,7,101,18,6]
0 1 2 3 4 5 6 7 8
1 1 1 2 2 3 1 1 1
2 3 7 101
2 3 101
25→②
1→1
2 3 7
2 5 7
for (i=0; i< arr.length
for (j=i-1; j>=0
if (arr(i) > arr(j)
x = dp(j)
dp(i) = max (dp(i), 1+x)

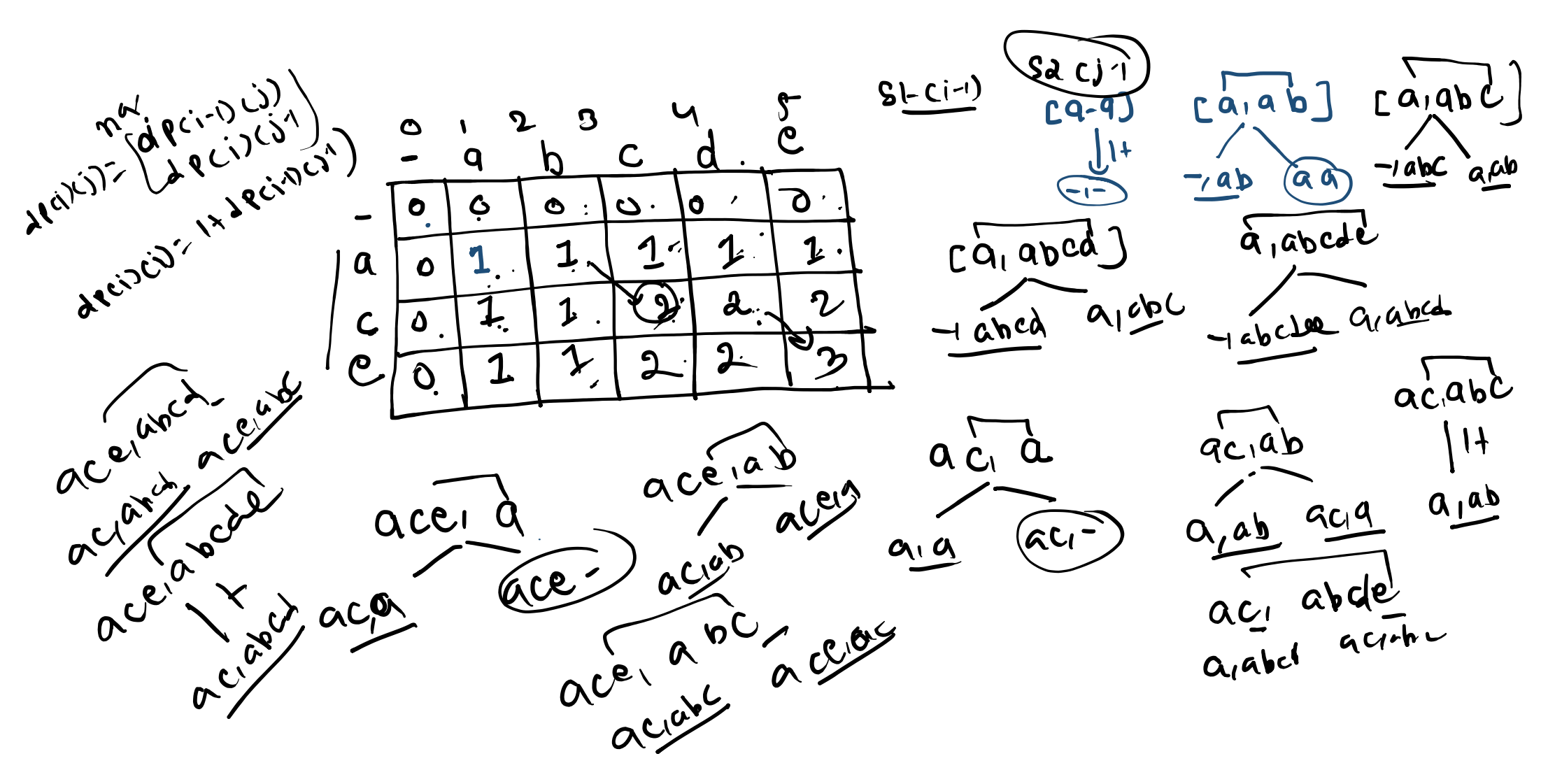


abcde, ace
1+2
(bcde, ce)
(cde, ce)
(bcde, e)
1+1
(de, e)
cde, e
ce
de, –
1+0
-1
2^n
2^1000

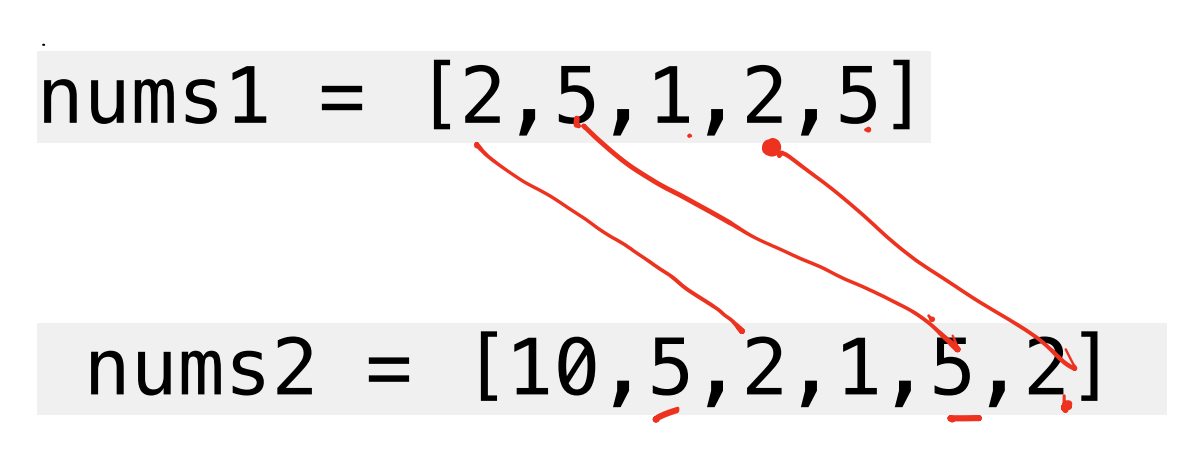


```
public static int LCS(String s1, String s2, int i, int j) {
    if (i == s1.length() || j == s2.length()) {
        return 0;
    }
    int ans = 0;
    if (s1.charAt(i) == s2.charAt(j)) {
        ans = 1 + LCS(s1, s2, i + 1, j + 1);
    } else {
        int f = LCS(s1, s2, i + 1, j);
        int s = LCS(s1, s2, i, j + 1);
        ans = Math.max(f, s);
    }
    return ans;
}
```

[abcde, ace] ③
1+2
[bcde, ce]
cde, ce
bcde, e
de, e
de –
cde, e
a c e
a b c d e

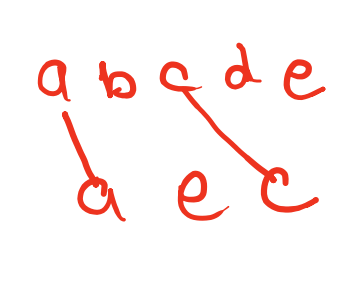


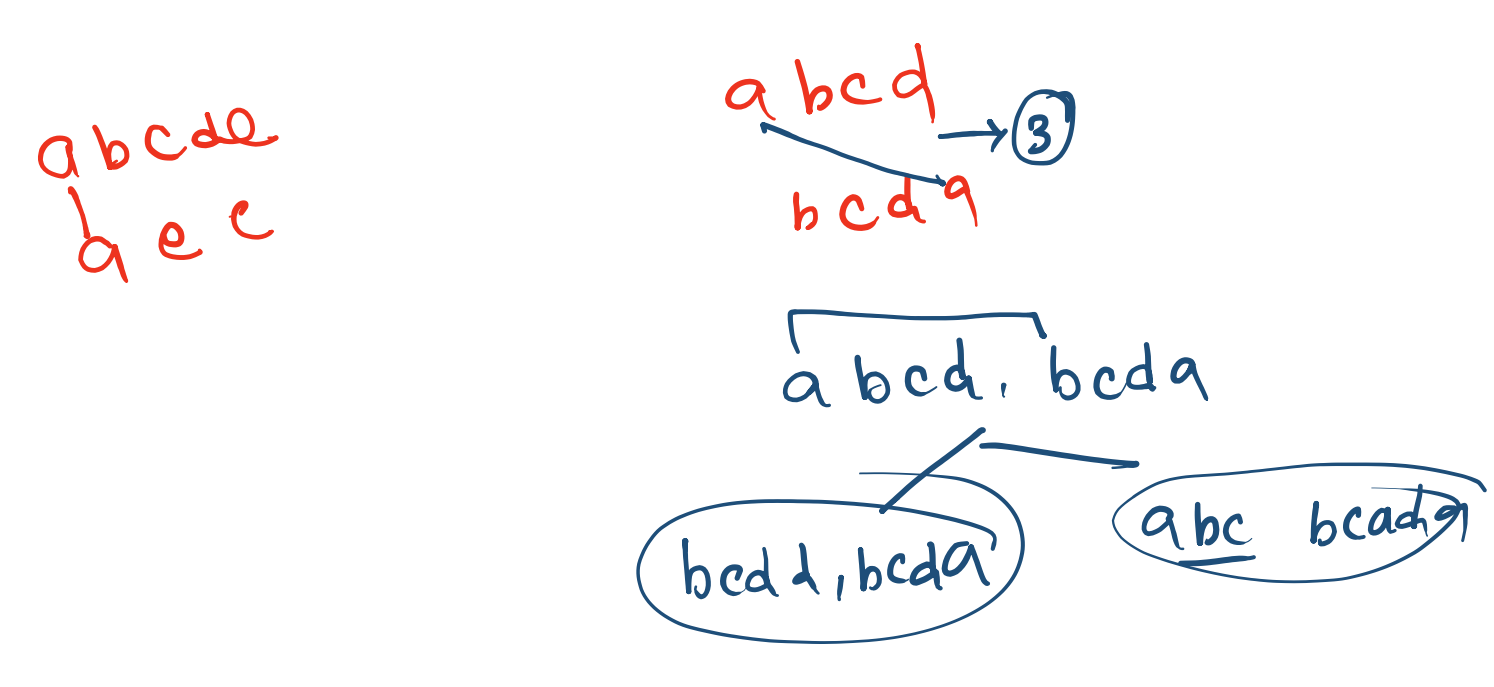
dp(i)(j) = max [dp(i-1)(j), dp(i)(j-1)]
dp(i)(j) = 1+dp(i-1)(j-1)
0 1 2 3 4 5
– a b c d e
a
c
e
s1 (i-1)
s2 (j-1)
[a, ab]
[a, abc]
a, abcd
ace, a
ace, abcde



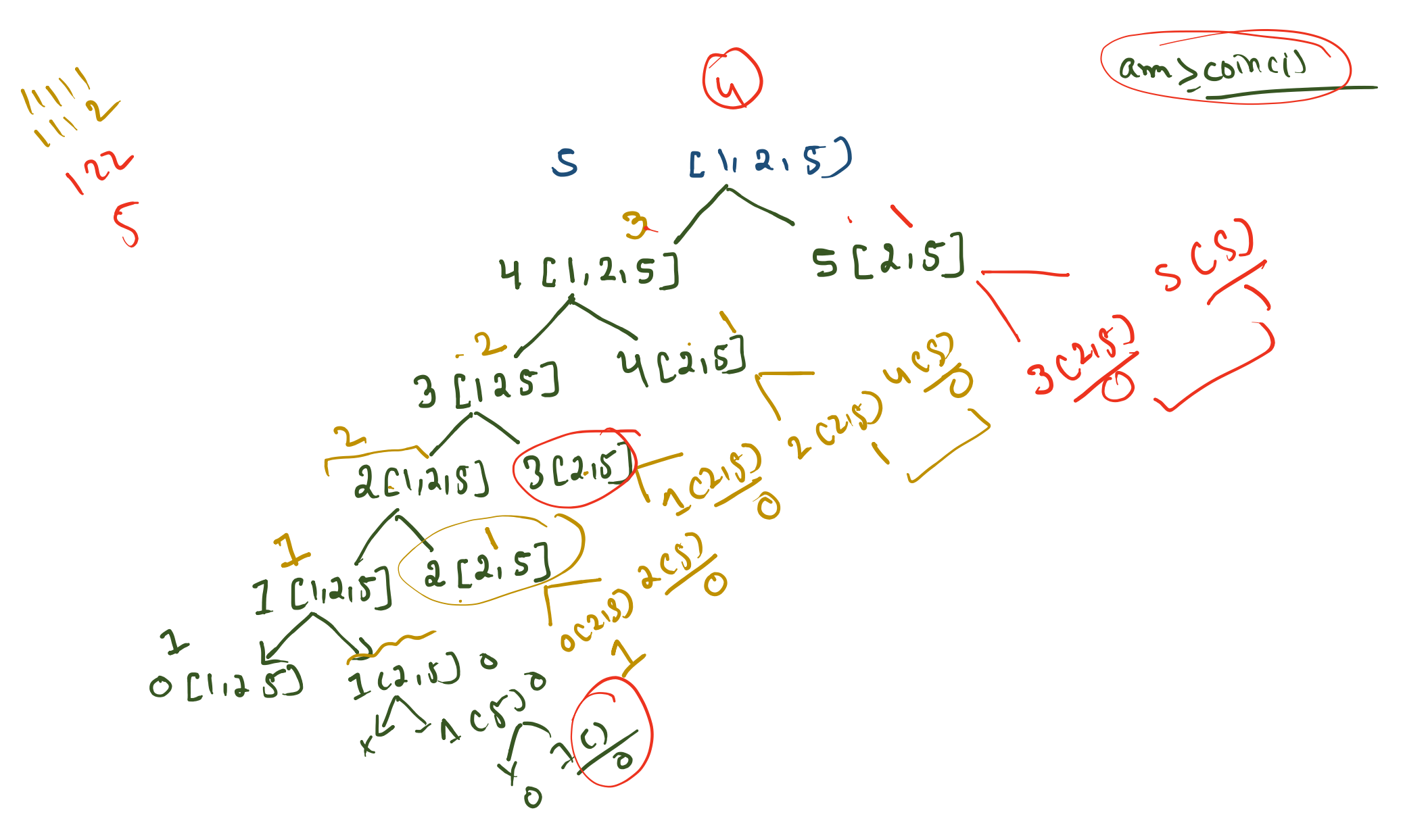
```
nums1 = [2,5,1,2,5]

nums2 = [10,5,2,1,5,2]
```

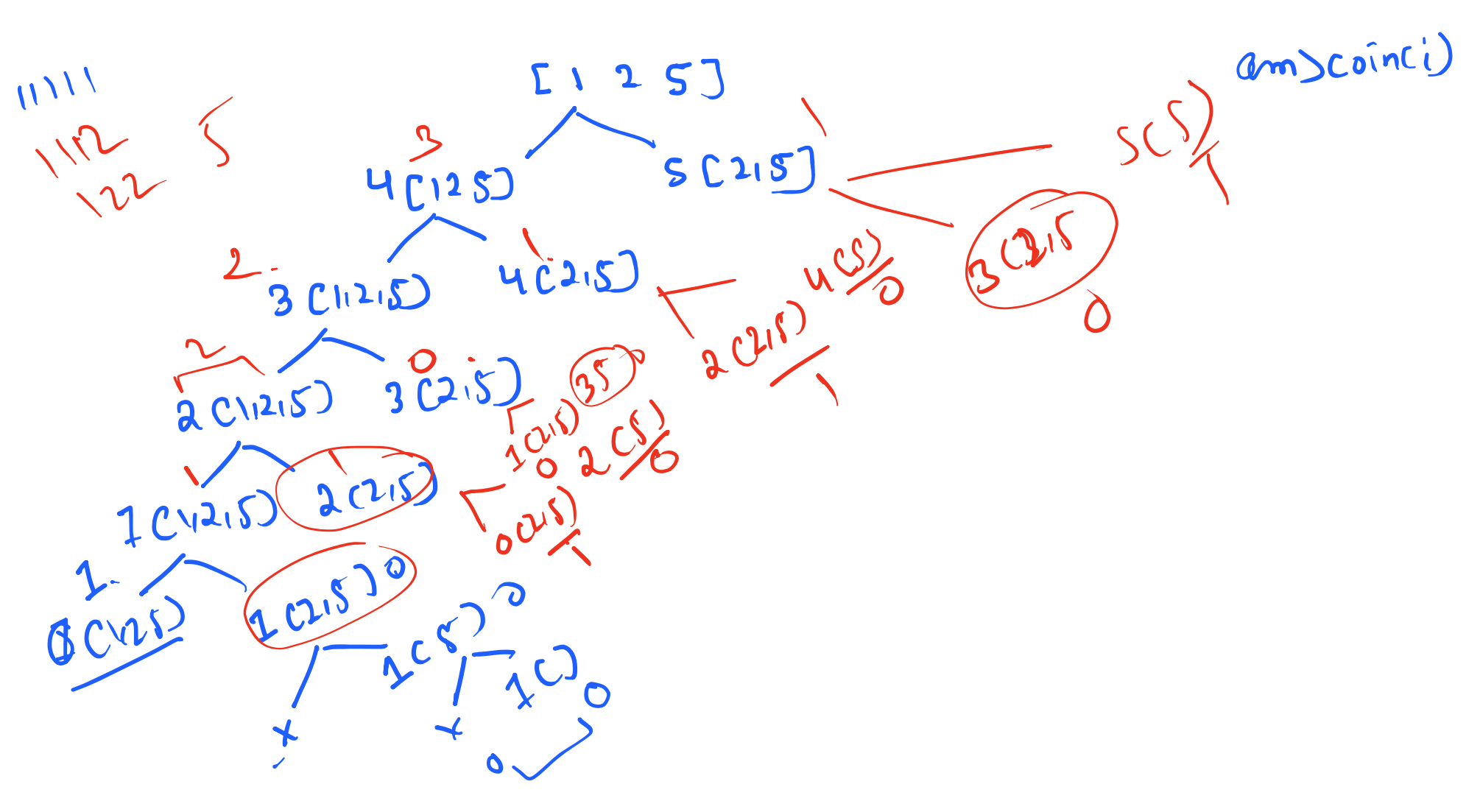




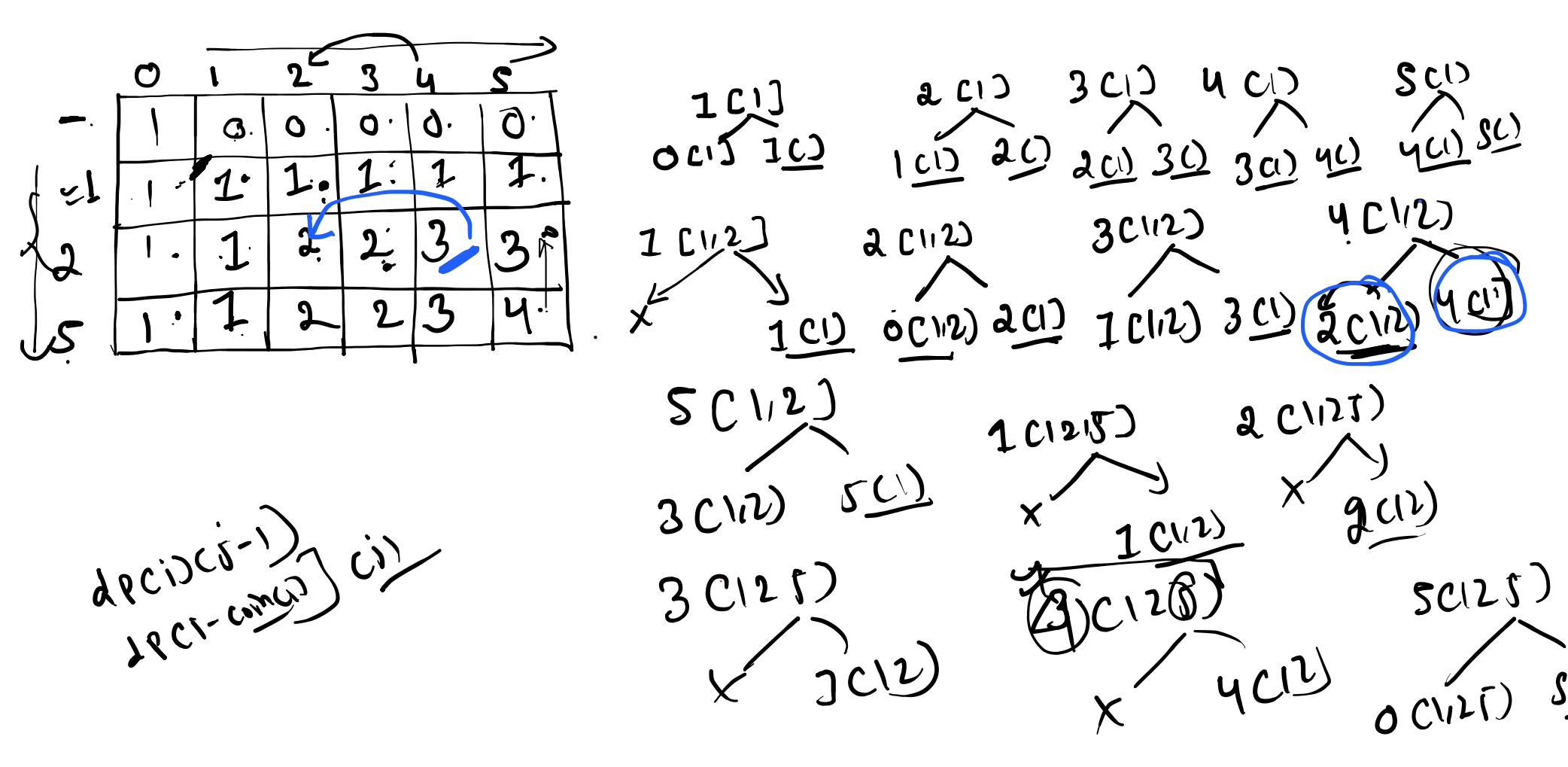
abcde
dec
abcd
bcda
③
abcd, bcda
bcd, bcda
abc bcda
am >= coin(i)



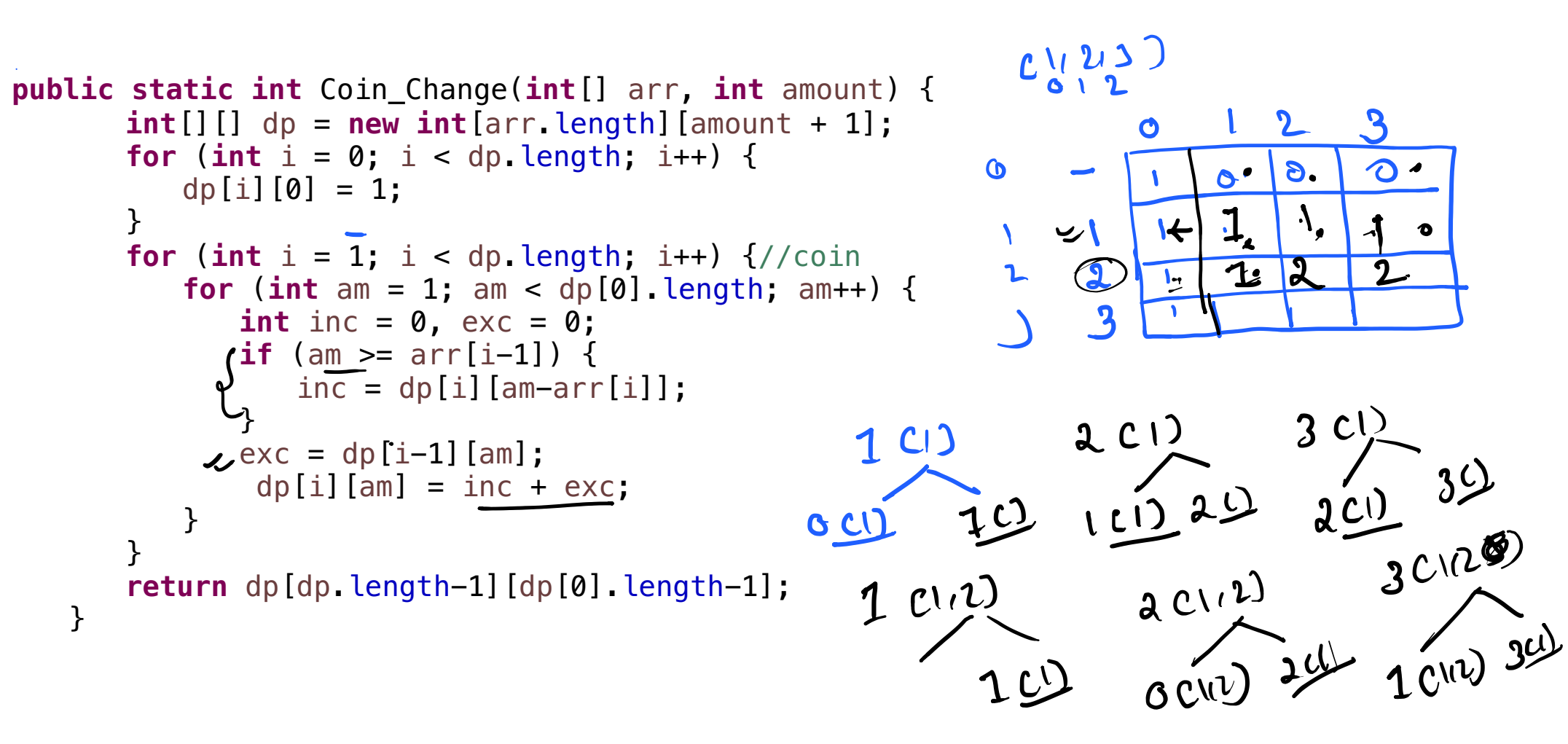
11111
1112
122
5
④
5 [1,2,5]
4 [1,2,5]
5 [2,5]
3 [1,2,5]
4 [2,5]
2 [1,2,5]
3 [2,5]
1 [1,2,5]
2 [2,5]
0 [1,2,5]
5 [5]



11111
1112
122
5
[1 2 5]
am >= coin(i)
4 [1 2 5]
5 [2 5]
3 [1,2,5]
4 [2,5]
2 [1,2,5]
3 [2,5]
1 [1,2,5]
2 [2,5]
0 [1,2,5]
1 [2,5] 0
5 (5)

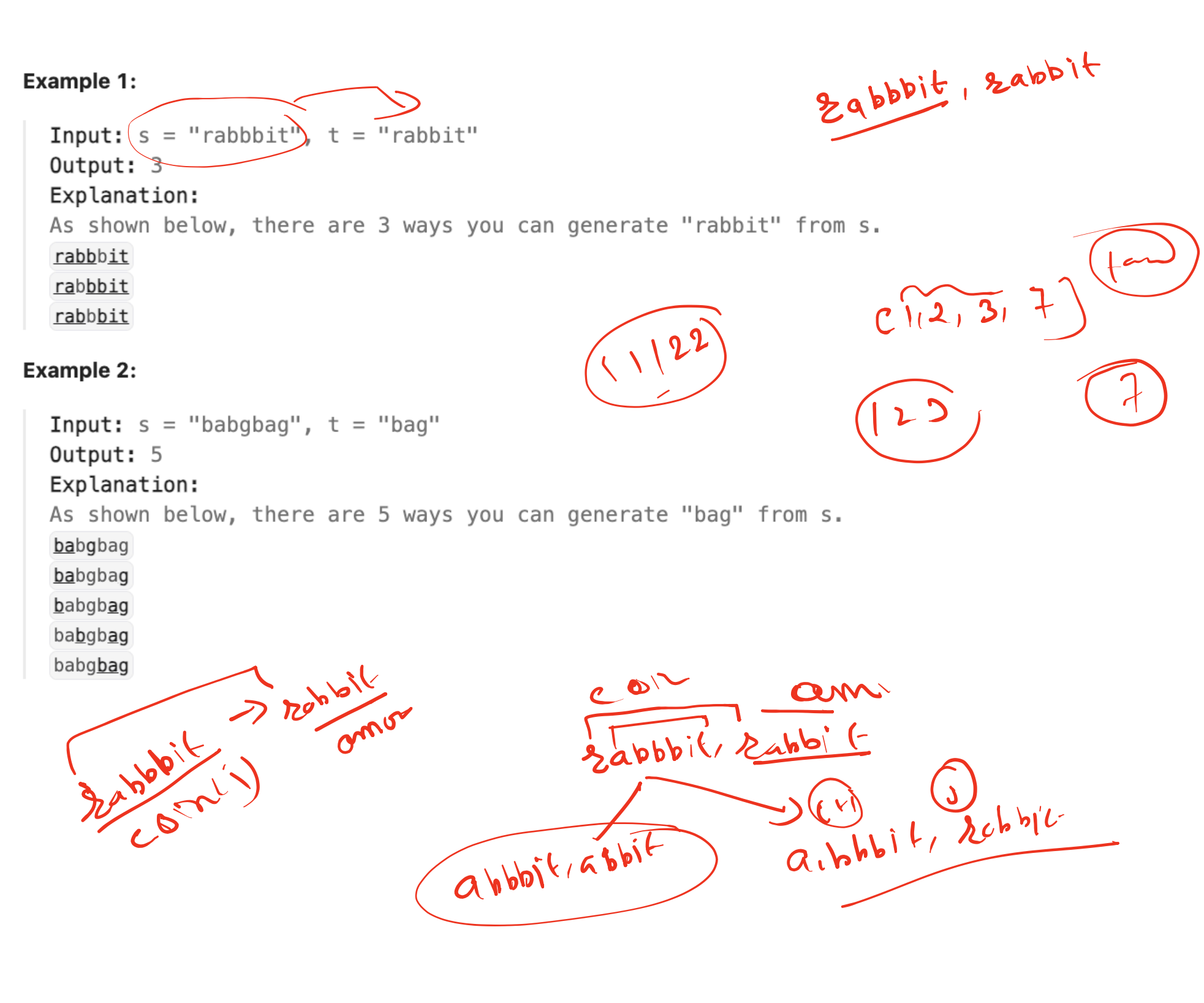


0 1 2 3 4 5
1 0 0 0 0 0
1 1 1 1 1 1
1 1 2 2 3 3
1 1 2 2 3 4
dp(i)(j-1)
dp(i-coin(j)) (j)
1 (1)
2 (1)
3 (1)
4 (1)
5 (1)
5 (1,2)
4 (1,2)
3 (1,2)
2 (1,2)
1 (1,2)
5 (1,2,5)



```
public static int Coin_Change(int[] arr, int amount) {
        int[][] dp = new int[arr.length][amount + 1];
        for (int i = 0; i < dp.length; i++) {
            dp[i][0] = 1;
        }
        for (int i = 1; i < dp.length; i++) {//coin
            for (int am = 1; am < dp[0].length; am++) {
                int inc = 0, exc = 0;
                if (am >= arr[i-1]) {
                    inc = dp[i][am-arr[i]];
                }
                exc = dp[i-1][am];
                dp[i][am] = inc + exc;
            }
        }
        return dp[dp.length-1][dp[0].length-1];
    }
```

c(1,2,3)
0 1 2 3
1 (1)
2 (1)
3 (1)
1 (1,2)
2 (1,2)
3 (1,2)



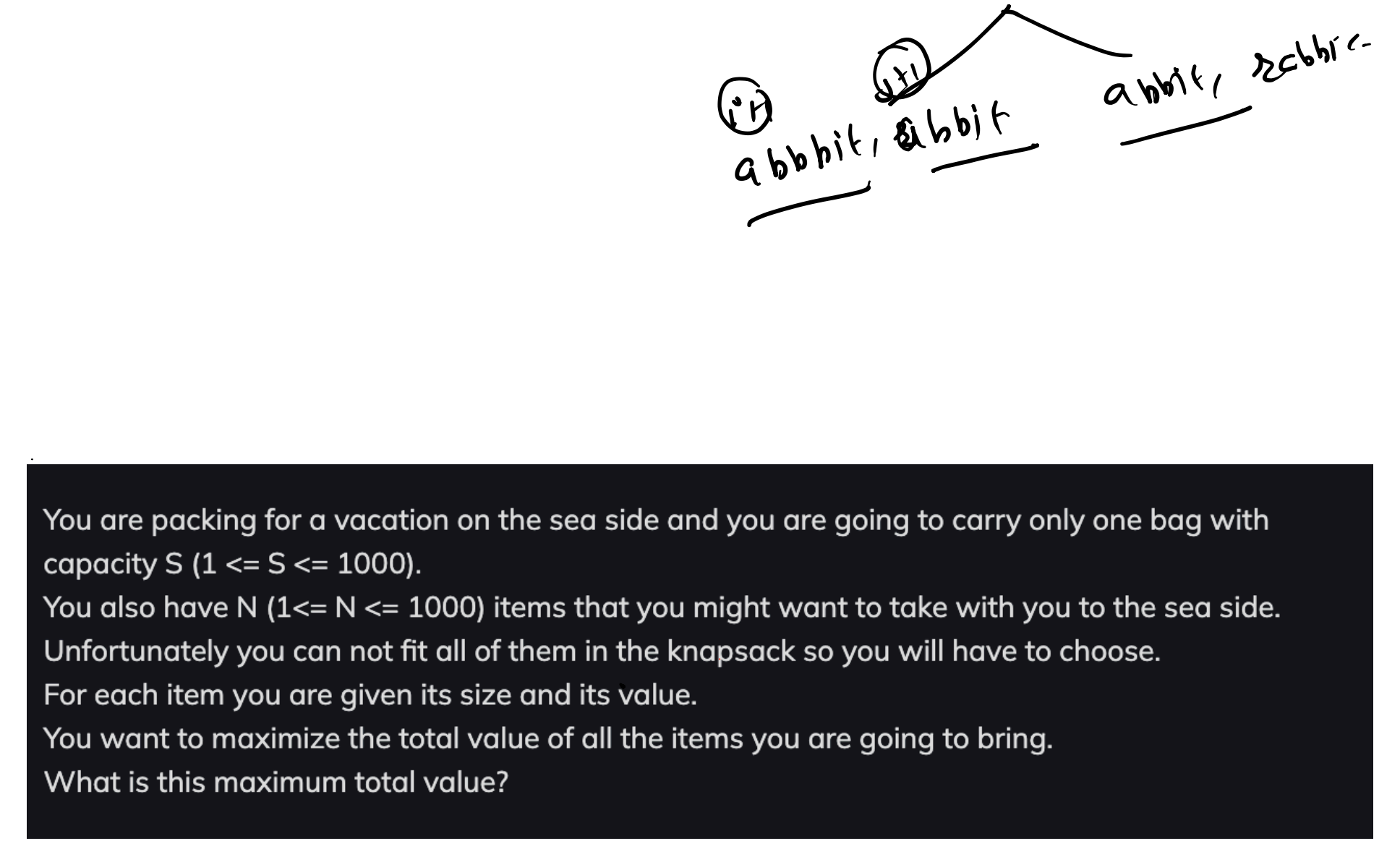
**Example 1:**

```
Input: s = "rabbbit", t = "rabbit"
Output: 3
Explanation:
As shown below, there are 3 ways you can generate "rabbit" from s.
rabbbit
rabbbit
rabbbit
```

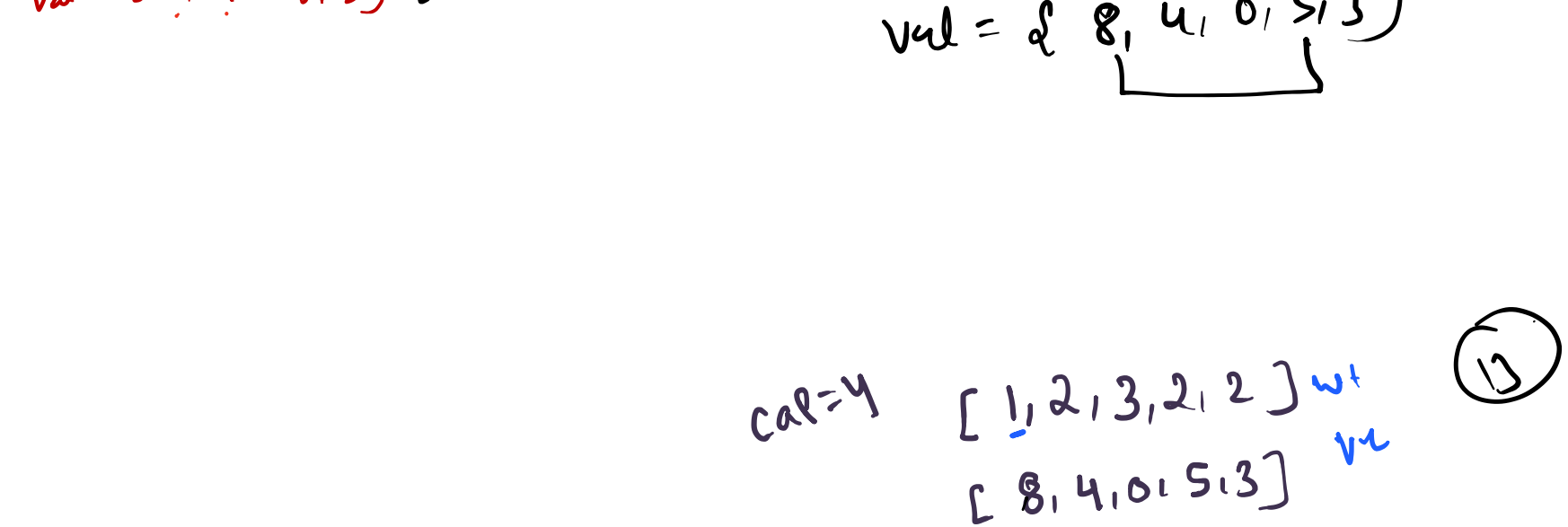
**Example 2:**

```
Input: s = "babgbag", t = "bag"
Output: 5
Explanation:
As shown below, there are 5 ways you can generate "bag" from s.
babgbag
babgbag
babgbag
babgbag
babgbag
```

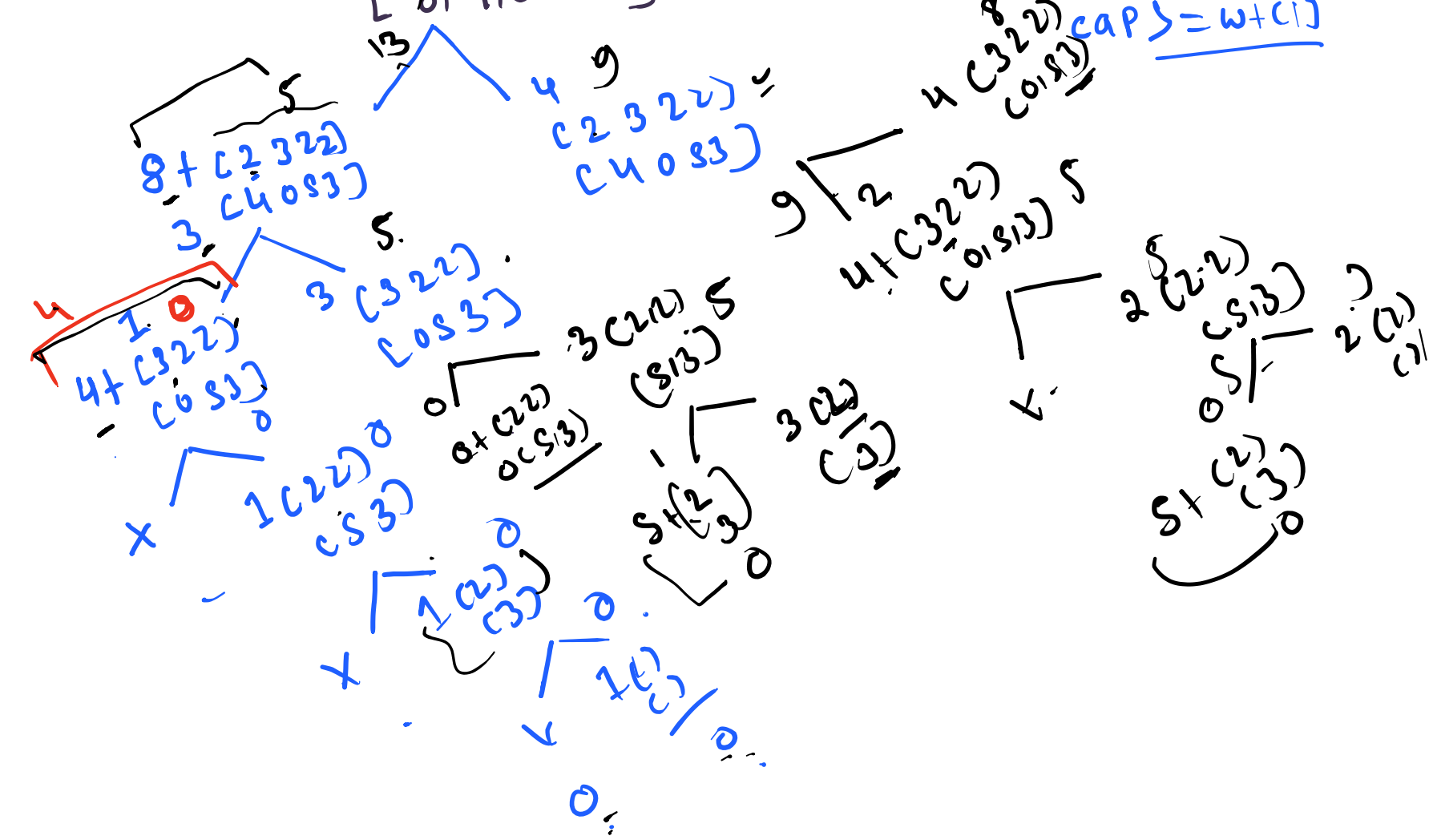
rabbbit, rabbit
(1,2,3,7)
(1122)
(123)
⑦
coin am
rabbbit, rabbit
abbbit, abbit
a, bbbit, rabbit



2 2 3
[1, 2, 3]
10
7 (1,2,3)
10 (1,2)
am >= coin(i)
rabbbit → rabbit
abbbit, abbit
abbit, rabbit



You are packing for a vacation on the sea side and you are going to carry only one bag with capacity S (1 <= S <= 1000).
You also have N (1<= N <= 1000) items that you might want to take with you to the sea side.
Unfortunately you can not fit all of them in the knapsack so you will have to choose.
For each item you are given its size and its value.
You want to maximize the total value of all the items you are going to bring.
What is this maximum total value?



cap = 4
N = 5
wt = [1, 2, 3, 2, 2]
val = [8, 4, 0, 5, 3]
8+0 = 8
5+3 = 8
8+4 = 12
8+5 = 13
cap = 4
[1, 2, 3, 2, 2] wt
[8, 4, 0, 5, 3] val
13
cap >= wt(i)